Cache vs cookies in Go

Contents

1. The one-line summary
2. Cookies in detail
3. Caching in detail
4. HTTP caching
5. Server-side caching in Go
6. Cookie vs cache: the actual differences
7. When to use which
8. Common patterns
9. Best practices
10. Common gotchas
11. Quick reference

Two web mechanisms that get mixed up in interviews. Both involve “stored data,” but they solve completely different problems. Here’s the difference, and how Go handles each.

1. The one-line summary

- **Cookies** are small pieces of data the server sends to the browser, which the browser sends back on every subsequent request. They identify the user.
- **Caching** is data stored somewhere (browser, CDN, server, app memory) to avoid repeating an expensive operation. It speeds things up.

Cookies = identity. Cache = speed.

2. Cookies in detail

What a cookie actually is

A cookie is a name=value pair, with optional attributes, transferred over HTTP headers.

```
Response: Set-Cookie: session=abc123; Path=/; HttpOnly; Secure; SameSite=Strict; Max-
Age=3600
Request: Cookie: session=abc123; theme=dark
```

The browser stores the cookie and attaches it to every matching request automatically. The server reads it from the `Cookie` header.

Limits:

- About 4 KB per cookie.
- About 50 cookies per domain.
- Domain-scoped (or path-scoped within a domain).

Cookie attributes

Attribute	Purpose
`Domain`	Which domain the browser sends it to
`Path`	Which URL prefix sends it
`Max-Age` / `Expires`	Lifetime; omitted means session cookie (gone on browser close)
`Secure`	Only sent over HTTPS
`HttpOnly`	JavaScript can't read it (XSS defense)
`SameSite`	Cross-site behavior: `Strict`, `Lax` (default in modern browsers), `None`
`Partitioned`	Per-top-level-site partition (CHIPS, newer)

`Secure`, `HttpOnly`, and `SameSite` are the security trio. Set them on anything sensitive (session IDs, CSRF tokens, auth tokens).

Cookies in Go: setting

```
import "net/http"

func handler(w http.ResponseWriter, r *http.Request) {
    cookie := &http.Cookie{
        Name:     "session",
        Value:    "abc123",
        Path:     "/",
        MaxAge:   3600,
        Secure:   true,
        HttpOnly: true,
        SameSite: http.SameSiteStrictMode,
    }
    http.SetCookie(w, cookie)
    w.Write([]byte("ok"))
}
```

`http.SetCookie` writes a Set-Cookie header. Call it before writing the body; once you start writing, headers are locked.

Cookies in Go: reading

```go
func handler(w http.ResponseWriter, r *http.Request) {
    c, err := r.Cookie("session")
    if err == http.ErrNoCookie {
        http.Error(w, "no session", http.StatusUnauthorized)
        return
    }
    if err != nil {
        http.Error(w, "bad cookie", http.StatusBadRequest)
        return
    }
    fmt.Println("session:", c.Value)
}

// or get all cookies
for _, c := range r.Cookies() {
    fmt.Println(c.Name, c.Value)
}
```

Deleting a cookie

There's no "delete" method. Overwrite with an expired one:

```go
http.SetCookie(w, &http.Cookie{
    Name:   "session",
    Value:  "",
    Path:   "/",
    MaxAge: -1,           // negative means delete now
})
```

`MaxAge: -1` translates to `Max-Age=0` and `Expires=Thu, 01 Jan 1970 ...` in the header, which most browsers treat as "remove."

Sessions: cookies + server-side store

A cookie that carries a session ID is paired with a server-side store (Redis, Postgres, etc.) holding the actual session data. The cookie is a pointer; the database is the value.

```
sessionID := generateRandomID()
redis.Set(ctx, "session:"+sessionID, userData, time.Hour)

http.SetCookie(w, &http.Cookie{
    Name:     "session",
    Value:    sessionID,
    HttpOnly: true,
    Secure:   true,
    SameSite: http.SameSiteStrictMode,
    MaxAge:   3600,
})
```

The alternative is a signed/encrypted cookie that carries the data itself (JWT, secure cookies). Tradeoffs:

	Server-side session	Client-side (JWT, secure cookie)
Storage cost	Server pays	Free for server
Revocation	Easy: delete in store	Hard: token valid until expiry
Size	Cookie tiny	Cookie large (every request carries it)
Read cost	DB lookup per request	Decode + verify signature
Best for	Web apps with revocation needs	Stateless APIs, microservices

Cookies vs Authorization header

For web apps, cookies are the default because the browser handles them automatically. For APIs called from JS / mobile / server, an `Authorization: Bearer <token>` header is common.

Cookies are vulnerable to CSRF (the browser attaches them automatically to cross-site requests). The `SameSite` attribute mitigates this, plus CSRF tokens for sensitive ops. Authorization headers aren't auto-attached, so they're CSRF-immune by default but need explicit handling.

3. Caching in detail

Caching means storing the result of an expensive operation so subsequent requests skip the work. The "expensive" part might be DB query time, network latency, CPU, or just byte count.

Where caches live:

Layer	Examples	Lifetime
Browser	HTTP cache from `Cache-Control`	days/weeks
CDN	Cloudflare, Fastly, CloudFront	minutes to days
Reverse proxy	Nginx, Varnish	seconds to hours
App memory	`map[K]V`, `sync.Map`, LRU library	request, minutes

Layer	Examples	Lifetime
Distributed	Redis, Memcached	minutes to hours
Database	query result cache, materialized view	varies

Each layer adds its own staleness/freshness tradeoff. Higher layers are faster but stale longer.

4. HTTP caching

The HTTP response headers tell clients (browser, CDN, intermediate proxies) how to cache.

Cache-Control

```
Cache-Control: public, max-age=3600
Cache-Control: private, no-cache
Cache-Control: no-store
```

Directives:

Directive	Meaning
`public`	Caches may store (CDN-friendly)
`private`	Only the end user's browser may store
`no-cache`	Cache, but revalidate every time (use ETag)
`no-store`	Don't cache at all
`max-age=N`	Fresh for N seconds
`s-maxage=N`	Same, but for shared caches (CDN) only
`must-revalidate`	When stale, revalidate before serving
`immutable`	Won't ever change (versioned static assets)

ETag and conditional requests

```
Response:   ETag: "v1-abc123"
Request:    If-None-Match: "v1-abc123"
Response:   304 Not Modified    (no body, save bandwidth)
```

ETag is a content fingerprint. The browser sends `If-None-Match` with the last ETag it saw. If the resource hasn't changed, the server replies 304 Not Modified with an empty body and the browser uses its cached copy.

`Last-Modified` + `If-Modified-Since` is the older equivalent using a timestamp.

Setting cache headers in Go

```
func staticHandler(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Cache-Control", "public, max-age=86400, immutable")
    w.Header().Set("ETag", `"v1-abc123"`)

    if match := r.Header.Get("If-None-Match"); match == `"v1-abc123"` {
        w.WriteHeader(http.StatusNotModified)
        return
    }

    serveFile(w, r)
}
```

For static files, `http.FileServer` doesn't add caching headers; wrap it in a middleware.

5. Server-side caching in Go

In-memory map

```
var (
    cache   = map[string]string{}
    cacheMu sync.RWMutex
)

func get(key string) (string, bool) {
    cacheMu.RLock()
    defer cacheMu.RUnlock()
    v, ok := cache[key]
    return v, ok
}

func set(key, value string) {
    cacheMu.Lock()
    defer cacheMu.Unlock()
    cache[key] = value
}
```

Problems with this naive version:

- No expiration: entries live forever.
- No size limit: grows unbounded.
- Single-process: each instance has its own copy.

LRU with expiration

`github.com/hashicorp/golang-lru/v2` is a common choice:

```
import "github.com/hashicorp/golang-lru/v2/expirable"

c := expirable.NewLRU[string, User](1000, nil, time.Minute*10)
c.Add("u-1", user)
v, ok := c.Get("u-1")
```

For single-process caching with bounds, that's typically enough.

Redis (distributed)

```
import "github.com/redis/go-redis/v9"

rdb := redis.NewClient(&redis.Options{Addr: "localhost:6379"})

err := rdb.Set(ctx, "user:1", json.Marshal(user), 10*time.Minute).Err()

val, err := rdb.Get(ctx, "user:1").Result()
if err == redis.Nil {
    // cache miss
}
```

Redis gives you shared cache across processes/hosts at the cost of a network roundtrip per access.

Cache-aside pattern

```
func getUser(ctx context.Context, id string) (*User, error) {
    if u, ok := lru.Get(id); ok {
        return u, nil
    }
    u, err := db.QueryUser(ctx, id)
    if err != nil { return nil, err }
    lru.Add(id, u)
    return u, nil
}
```

The application reads-through and writes-through the cache. Other patterns: write-back, write-around, read-through (where the cache itself fetches on miss).

Cache invalidation strategies

- TTL only. Set an expiration; accept staleness up to that bound.
- Explicit invalidation. On every write, delete or update the cached entry.
- Versioned keys. Append a version: `user:1:v3`. Bump the version to invalidate.
- Event-driven. Pub/sub on writes; subscribers invalidate.

Cache invalidation is famously hard. Most production caches use TTL plus explicit invalidation for hot keys.

Singleflight (avoiding thundering herd)

When 100 concurrent requests all miss the cache for the same key, you don't want 100 DB queries. Use `singleflight`:

```
import "golang.org/x/sync/singleflight"

var group singleflight.Group

func getUser(id string) (*User, error) {
    v, err, _ := group.Do(id, func() (any, error) {
        return loadFromDB(id)
    })
    return v.(*User), err
}
```

The first call runs the function; concurrent callers wait and share the result. Cuts thundering herd to one query per concurrent batch.

6. Cookie vs cache: the actual differences

Question	Cookie	Cache
Who stores it	Browser (sent back to server)	Anywhere (browser, CDN, server, distributed store)
Sent on every request	Yes (automatically)	No
Size	About 4 KB	Bounded by storage, not by request size
Purpose	Identify user / store small state	Avoid redoing expensive work
Lifetime	Set by Max-Age or session	Set by TTL, cache size, or invalidation policy
Modified by	Server (Set-Cookie)	Anyone with access to the cache
Security concerns	XSS, CSRF, session fixation	Stale data, cache poisoning
Goes through proxies	Yes (private data!)	Sometimes (HTTP cache layer)
In Go	`net/http.Cookie`	LRU lib, Redis, plain map, HTTP headers

7. When to use which

Cookies: anything tied to "who is making this request."

- Session ID
- Auth token (web apps)

- CSRF token
- User preferences (theme, language)
- Tracking / analytics consent

Caching: anything where “I just computed this and it’s still valid.”

- DB query results (especially read-heavy)
- API responses to external services
- Rendered HTML pages or fragments
- Computed aggregations
- Static assets (CSS, JS, images)

8. Common patterns

Login flow (cookies)

1. POST /login {user, pass}
2. Server validates, generates session ID
3. Server stores session in Redis: `SET session:abc { userID: 42, exp: ... }`
4. Server responds: `Set-Cookie: session=abc; HttpOnly; Secure; SameSite=Strict`
5. Browser sends `Cookie: session=abc` on subsequent requests
6. Server reads cookie, looks up session in Redis, identifies the user

Read-heavy API endpoint (cache)

1. Client requests GET /api/products/123
2. Handler checks LRU cache for key `"product:123"`
3. Miss: query DB, store in cache with 5-minute TTL, return
4. Hit: return from cache (no DB query)
5. On product update: delete cache key `"product:123"`

Static asset (HTTP cache)

1. Client requests /assets/app.abc123.js
2. Server responds with body + `Cache-Control: public, max-age=31536000, immutable`
3. Browser caches for 1 year
4. On code deploy, change filename to /assets/app.def456.js
5. Browser sees new URL, fetches new file; old cached file ignored

9. Best practices

Cookies

1. Always set `HttpOnly` on session cookies. XSS-stolen sessions are catastrophic.
2. Set `Secure` in production. Don't transmit auth cookies over HTTP.
3. Set `SameSite=Lax` minimum, `Strict` for sensitive endpoints. CSRF defense.
4. Use server-side sessions for revocation needs. Pure JWT means you can't kick someone out before token expiry.
5. Rotate session IDs after login. Prevents session fixation.
6. Encrypt sensitive cookie contents (`gorilla/securecookie` , etc.) if you must store them client-side.
7. Keep cookies small. Every request carries them.

Caching

1. Always set a TTL. "Forever" caches become memory leaks.
2. Cap the cache size. Use an LRU; don't use a raw map for hot loops.
3. Cache the answer, not the inputs. Computed result is what you want to reuse.
4. Use singleflight for expensive misses to avoid thundering herd.
5. Invalidate aggressively when the truth changes. Stale data is worse than slow data for most apps.
6. Set `Cache-Control` correctly on responses. Don't leak private data into shared caches.
7. Use ETag for large, occasionally-changing resources. Saves bandwidth even when the body changes.
8. Cache keys must include all variables. A cache keyed only by user ID but varying by locale is a correctness bug.

10. Common gotchas

Gotcha	Symptom	Fix
Set-Cookie after writing body	Header ignored	Set cookies first
Cookie sent on cross-site request	CSRF	`SameSite=Lax` or `Strict`
JWT can't be revoked	Logged-out users still authenticated	Keep short expiry; use refresh tokens or session store
Cache stale forever after data change	Wrong answers	TTL + explicit invalidation
Single map cache grows forever	Memory leak	LRU with cap
Thundering herd on cache miss	DB load spikes	singleflight
Caching responses with auth headers	Leaks across users	`Cache-Control: private`

Gotcha	Symptom	Fix
Wrong cache key (forgot a dimension)	Wrong data served	Include all varying inputs in the key
Stale cache after deploy	Users see old assets	Versioned filenames + immutable
Cookie size > 4 KB	Browser drops or truncates	Use server-side session
HTTP cache pollution from query strings	Multiple entries for same content	Strip irrelevant params before cache key
Reading cookies in handler chain	Each `r.Cookie()` re-parses	Cache the lookup once

11. Quick reference

What	Tool / API
Set cookie	`http.SetCookie(w, &http.Cookie{...})`
Read cookie	`r.Cookie("name")`
Read all cookies	`r.Cookies()`
Delete cookie	Re-set with `MaxAge: -1`
Strict cookie defaults	`HttpOnly: true, Secure: true, SameSite: SameSiteStrictMode`
HTTP cache header	`w.Header().Set("Cache-Control", "public, max-age=3600")`
ETag	`w.Header().Set("ETag", "...")` and check `If-None-Match`
In-process LRU	`github.com/hashicorp/golang-lru/v2`
LRU with TTL	`golang-lru/v2/expirable`
Distributed cache	`github.com/redis/go-redis/v9`
Avoid thundering herd	`golang.org/x/sync/singleflight`
Encrypted client-side cookie	`github.com/gorilla/securecookie`
Session library	`github.com/gorilla/sessions`
Static immutable URL	`app.<hash>.js` + `Cache-Control: immutable`